

**Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

ОТЧЕТ

ПО ЛАБОРАТОРНЫМ РАБОТАМ № 6

**«Работа с БД в СУБД MongoDB»
по дисциплине «Проектирование и реализация баз данных»**

Обучающийся Кретов Иван Алексеевич

Факультет прикладной информатики

Группа К3239

Направление подготовки 09.03.03 Прикладная информатика

Образовательная программа Мобильные и сетевые технологии 2024

Преподаватели Говорова Марина Михайловна, Белов Александр Олегович

Санкт-Петербург
2025/2026

Цель работы: овладеть практическими навыками работы с CRUD-операциями, с вложенными объектами в коллекции базы данных MongoDB, агрегации и изменения данных, со ссылками и индексами в базе данных MongoDB.

Выполнение:

Практическое задание 2.1.1:

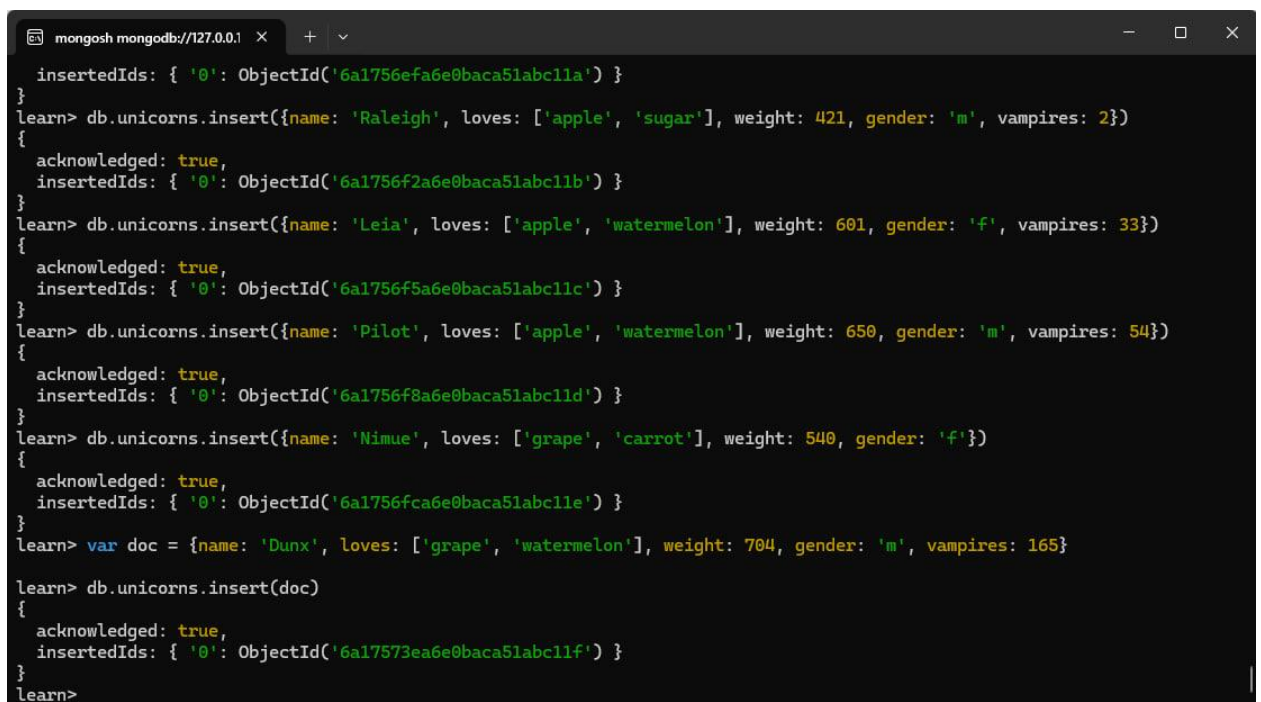
1. *Создайте базу данных learn.*
2. *Заполните коллекцию единорогов unicorns:*

```
db.unicorns.insert({name: 'Horny', loves: ['carrot', 'papaya'], weight: 600, gender: 'm', vampires: 63});
db.unicorns.insert({name: 'Aurora', loves: ['carrot', 'grape'], weight: 450, gender: 'f', vampires: 43});
db.unicorns.insert({name: 'Unicrom', loves: ['energon', 'redbull'], weight: 984, gender: 'm', vampires: 182});
db.unicorns.insert({name: 'Rooooooodles', loves: ['apple'], weight: 575, gender: 'm', vampires: 99});
db.unicorns.insert({name: 'Solnara', loves: ['apple', 'carrot', 'chocolate'], weight: 550, gender: 'f', vampires: 80});
db.unicorns.insert({name: 'Ayna', loves: ['strawberry', 'lemon'], weight: 733, gender: 'f', vampires: 40});
db.unicorns.insert({name: 'Kenny', loves: ['grape', 'lemon'], weight: 690, gender: 'm', vampires: 39});
db.unicorns.insert({name: 'Raleigh', loves: ['apple', 'sugar'], weight: 421, gender: 'm', vampires: 2});
db.unicorns.insert({name: 'Leia', loves: ['apple', 'watermelon'], weight: 601, gender: 'f', vampires: 33});
db.unicorns.insert({name: 'Pilot', loves: ['apple', 'watermelon'], weight: 650, gender: 'm', vampires: 54});
db.unicorns.insert({name: 'Nimue', loves: ['grape', 'carrot'], weight: 540, gender: 'f'});
```

3. *Используя второй способ, вставьте в коллекцию единорогов документ:*

```
{name: 'Dunx', loves: ['grape', 'watermelon'], weight: 704, gender: 'm', vampires: 165}
```
4. *Проверьте содержимое коллекции с помощью метода find.*

db.unicorns.find()



```
mongosh mongodb://127.0.0.1 x + v
insertedIds: { '0': ObjectId('6a1756efa6e0baca51abc11a') }
}
learn> db.unicorns.insert({name: 'Raleigh', loves: ['apple', 'sugar'], weight: 421, gender: 'm', vampires: 2})
{
  acknowledged: true,
  insertedIds: { '0': ObjectId('6a1756f2a6e0baca51abc11b') }
}
learn> db.unicorns.insert({name: 'Leia', loves: ['apple', 'watermelon'], weight: 601, gender: 'f', vampires: 33})
{
  acknowledged: true,
  insertedIds: { '0': ObjectId('6a1756f5a6e0baca51abc11c') }
}
learn> db.unicorns.insert({name: 'Pilot', loves: ['apple', 'watermelon'], weight: 650, gender: 'm', vampires: 54})
{
  acknowledged: true,
  insertedIds: { '0': ObjectId('6a1756f8a6e0baca51abc11d') }
}
learn> db.unicorns.insert({name: 'Nimue', loves: ['grape', 'carrot'], weight: 540, gender: 'f'})
{
  acknowledged: true,
  insertedIds: { '0': ObjectId('6a1756fca6e0baca51abc11e') }
}
learn> var doc = {name: 'Dunx', loves: ['grape', 'watermelon'], weight: 704, gender: 'm', vampires: 165}
learn> db.unicorns.insert(doc)
{
  acknowledged: true,
  insertedIds: { '0': ObjectId('6a17573ea6e0baca51abc11f') }
}
learn>
```

```
mongosh mongodb://127.0.0.1 x + v
learn> db.unicorns.find()
[
  {
    _id: ObjectId('6a175699a6e0baca51abc114'),
    name: 'Horny',
    loves: [ 'carrot', 'papaya' ],
    weight: 600,
    gender: 'm',
    vampires: 63
  },
  {
    _id: ObjectId('6a1756c2a6e0baca51abc115'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  },
  {
    _id: ObjectId('6a1756e1a6e0baca51abc116'),
    name: 'Unicrom',
    loves: [ 'energon', 'redbull' ],
    weight: 984,
    gender: 'm',
    vampires: 182
  },
  {
    _id: ObjectId('6a1756e6a6e0baca51abc117'),
    name: 'Roooooodles',
    loves: [ 'apple' ],

```

Практическое задание 2.2.1:

1. Сформируйте запросы для вывода списков самцов и самок единорогов. Ограничьте список самок первыми тремя особями. Отсортируйте списки по имени.
2. Найдите всех самок, которые любят carrot. Ограничьте этот список первой особью с помощью функций `findOne` и `limit`.

```
db.unicorns.find({gender: 'm'}).sort({name: 1})
db.unicorns.find({gender: 'f'}).sort({name: 1}).limit(3)
db.unicorns.findOne({gender: 'f', loves: 'carrot'})
db.unicorns.find({gender: 'f', loves: 'carrot'}).limit(1)
```

```
learn> db.unicorns.find({gender: 'm'}).sort({name: 1})
```

```
[
  {
    _id: ObjectId('6a17573ea6e0baca51abc11f'),
    name: 'Dunx',
    loves: [ 'grape', 'watermelon' ],
    weight: 704,
    gender: 'm',
    vampires: 165
  },
  {
    _id: ObjectId('6a175699a6e0baca51abc114'),
    name: 'Horny',
    loves: [ 'carrot', 'papaya' ],
    weight: 600,
    gender: 'm',
    vampires: 63
  },
  {
    _id: ObjectId('6a1756efa6e0baca51abc11a'),
    name: 'Kenny',
    loves: [ 'grape', 'lemon' ],
    weight: 690,
    gender: 'm',
    vampires: 39
  },
  {
    _id: ObjectId('6a1756f8a6e0baca51abc11d'),
    name: 'Pilot',
    loves: [ 'apple', 'watermelon' ],
```

```
learn> db.unicorns.find({gender: 'f'}).sort({name: 1}).limit(3)
[
  {
    _id: ObjectId('6a1756c2a6e0baca51abc115'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  },
  {
    _id: ObjectId('6a1756eca6e0baca51abc119'),
    name: 'Ayna',
    loves: [ 'strawberry', 'lemon' ],
    weight: 733,
    gender: 'f',
    vampires: 40
  },
  {
    _id: ObjectId('6a1756f5a6e0baca51abc11c'),
    name: 'Leia',
    loves: [ 'apple', 'watermelon' ],
    weight: 601,
    gender: 'f',
    vampires: 33
  }
]
learn> |
```

```
learn> db.unicorns.findOne({gender: 'f', loves: 'carrot'})
{
  _id: ObjectId('6a1756c2a6e0baca51abc115'),
  name: 'Aurora',
  loves: [ 'carrot', 'grape' ],
  weight: 450,
  gender: 'f',
  vampires: 43
}
learn> db.unicorns.find({gender: 'f', loves: 'carrot'}).limit(1)
[
  {
    _id: ObjectId('6a1756c2a6e0baca51abc115'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  }
]
learn> |
```

Практическое задание 2.2.2:

Модифицируйте запрос для вывода списков самцов единорогов, исключив из результата информацию о предпочтениях и поле.

```
db.unicorns.find({gender: 'm'}, {loves: 0, gender: 0})
```

```
learn> db.unicorns.find({gender: 'm'}, {loves: 0, gender: 0})
[
  {
    _id: ObjectId('6a175699a6e0baca51abc114'),
    name: 'Horny',
    weight: 600,
    vampires: 63
  },
  {
    _id: ObjectId('6a1756e1a6e0baca51abc116'),
    name: 'Unicrom',
    weight: 984,
    vampires: 182
  },
  {
    _id: ObjectId('6a1756e6a6e0baca51abc117'),
    name: 'Roooooodles',
    weight: 575,
    vampires: 99
  },
  {
    _id: ObjectId('6a1756efa6e0baca51abc11a'),
    name: 'Kenny',
    weight: 690,
    vampires: 39
  },
  {
    _id: ObjectId('6a1756f2a6e0baca51abc11b'),
```

Практическое задание 2.2.3:

Вывести список единорогов в обратном порядке добавления.

```
db.unicorns.find().sort({$natural: -1})
```



```
learn> db.unicorns.find().sort({$natural: -1})
[
  {
    _id: ObjectId('6a17573ea6e0baca51abc11f'),
    name: 'Dunx',
    loves: [ 'grape', 'watermelon' ],
    weight: 704,
    gender: 'm',
    vampires: 165
  },
  {
    _id: ObjectId('6a1756fca6e0baca51abc11e'),
    name: 'Nimue',
    loves: [ 'grape', 'carrot' ],
    weight: 540,
    gender: 'f'
  },
  {
    _id: ObjectId('6a1756f8a6e0baca51abc11d'),
    name: 'Pilot',
    loves: [ 'apple', 'watermelon' ],
    weight: 650,
    gender: 'm',
    vampires: 54
  },
  {
    _id: ObjectId('6a1756f5a6e0baca51abc11c'),
    name: 'Leia',
    loves: [ 'apple', 'watermelon' ],
    weight: 601,

```

Практическое задание 2.2.4:

Вывести список единорогов с названием первого любимого предпочтения, исключив идентификатор.

```
db.unicorns.find({}, {_id: 0, name: 1, loves: {$slice: 1}})
```



```
learn> db.unicorns.find({}, {_id: 0, loves: {$slice: 1}})
[
  {
    name: 'Horny',
    loves: [ 'carrot' ],
    weight: 600,
    gender: 'm',
    vampires: 63
  },
  {
    name: 'Aurora',
    loves: [ 'carrot' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  },
  {
    name: 'Unicrom',
    loves: [ 'energon' ],
    weight: 984,
    gender: 'm',
    vampires: 182
  },
  {
    name: 'Rooooooodles',
    loves: [ 'apple' ],
    weight: 575,
    gender: 'm',

```

Практическое задание 2.3.1:

Вывести список самок единорогов весом от полутонны до 700 кг, исключив вывод идентификатора.

```
db.unicorns.find({gender: 'f', weight: {$gte: 500, $lte: 700}}, {_id: 0})
```

```
learn> db.unicorns.find({gender: 'f', weight: {$gte: 500, $lte: 700}}, {_id: 0})
[
  {
    name: 'Solnara',
    loves: [ 'apple', 'carrot', 'chocolate' ],
    weight: 550,
    gender: 'f',
    vampires: 80
  },
  {
    name: 'Leia',
    loves: [ 'apple', 'watermelon' ],
    weight: 601,
    gender: 'f',
    vampires: 33
  },
  {
    name: 'Nimue',
    loves: [ 'grape', 'carrot' ],
    weight: 540,
    gender: 'f'
  }
]
```

Практическое задание 2.3.2:

Вывести список самцов единорогов весом от полутонны и предпочитающих grape и lemon, исключив вывод идентификатора.

```
db.unicorns.find({gender: 'm', weight: {$gte: 500}, loves: {$all: ['grape', 'lemon']}, {_id: 0})
```

```
learn> db.unicorns.find({gender: 'm', weight: {$gte: 500}, loves: {$all: ['grape', 'lemon']}, {_id: 0})
[
  {
    name: 'Kenny',
    loves: [ 'grape', 'lemon' ],
    weight: 690,
    gender: 'm',
    vampires: 39
  }
]
```

Практическое задание 2.3.3:

Найти всех единорогов, не имеющих ключ vampires.

```
db.unicorns.find({vampires: {$exists: false}})
```

```
learn> db.unicorns.find({vampires: {$exists: false}})
[
  {
    _id: ObjectId('6a1756fca6e0baca51abc11e'),
    name: 'Nimue',
    loves: [ 'grape', 'carrot' ],
    weight: 540,
    gender: 'f'
  }
]
```

Практическое задание 2.3.4:

Вывести список упорядоченный список имен самцов единорогов с информацией об их первом предпочтении.

```
db.unicorns.find({gender: 'm'}, {_id: 0, name: 1, loves: {$slice: 1}}).sort({name: 1})
```

```
learn> db.unicorns.find({gender: 'm'}, {_id: 0, name: 1, loves: {$slice: 1}}).sort({name: 1})
[
  { name: 'Dunx', loves: [ 'grape' ] },
  { name: 'Horny', loves: [ 'carrot' ] },
  { name: 'Kenny', loves: [ 'grape' ] },
  { name: 'Pilot', loves: [ 'apple' ] },
  { name: 'Raleigh', loves: [ 'apple' ] },
  { name: 'Roooooodles', loves: [ 'apple' ] },
  { name: 'Unicrom', loves: [ 'energon' ] }
]
```

Практическое задание 3.1.1:

1. *Создайте коллекцию towns, включающую следующие документы:*

```
{name: "Punxsutawney ",
populatiuon: 6200,
last_sensus: ISODate("2008-01-31"),
famous_for: [""],
mayor: {
  name: "Jim Wehrle"
}}

{name: "New York",
populatiuon: 22200000,
last_sensus: ISODate("2009-07-31"),
famous_for: ["status of liberty", "food"],
mayor: {
  name: "Michael Bloomberg",
  party: "I"}}

{name: "Portland",
populatiuon: 528000,
last_sensus: ISODate("2009-07-20"),
famous_for: ["beer", "food"],
mayor: {
  name: "Sam Adams",
  party: "D"}}
```

2. *Сформировать запрос, который возвращает список городов с независимыми мэрами (party="I") . Вывести только название города и информацию о мэре.*

3. *Сформировать запрос, который возвращает список беспартийных мэров (party отсутствует) . Вывести только название города и информацию о мэре.*

```
db.towns.find({"mayor.party": "I"}, {_id: 0, name: 1, mayor: 1})
```

```
db.towns.find({"mayor.party": {$exists: false}}, {_id: 0, name: 1, mayor: 1})
```

```

learn> db.towns.insert({
|   name: "Punxsutawney",
|   population: 6200,
|   last_census: ISODate("2008-01-31"),
|   famous_for: [""],
|   mayor: { name: "Jim Wehrle" }
| })
DeprecationWarning: Collection.insert() is deprecated. Use insertOne
{
  acknowledged: true,
  insertedIds: { '0': ObjectId('6a175b6d2f0fef2858abc114') }
}
learn> db.towns.insert({
|   name: "New York",
|   population: 22200000,
|   last_census: ISODate("2009-07-31"),
|   famous_for: ["statue of liberty", "food"],
|   mayor: { name: "Michael Bloomberg", party: "I" }
| })
{
  acknowledged: true,
  insertedIds: { '0': ObjectId('6a175b782f0fef2858abc115') }
}
learn> db.towns.insert({
|   name: "Portland",
|   population: 528000,
|   last_census: ISODate("2009-07-20"),
|   famous_for: ["beer", "food"],
|   mayor: { name: "Sam Adams", party: "D" }
| })

learn> db.towns.find({"mayor.party": "I"}, {_id: 0, name: 1, mayor: 1})
[
  { name: 'New York', mayor: { name: 'Michael Bloomberg', party: 'I' } }
]
learn> db.towns.find({"mayor.party": {$exists: false}}, {_id: 0, name: 1, mayor: 1})
[ { name: 'Punxsutawney', mayor: { name: 'Jim Wehrle' } } ]

```

Практическое задание 3.1.2:

1. Сформировать функцию для вывода списка самцов единорогов.
2. Создать курсор для этого списка из первых двух особей с сортировкой в лексикографическом порядке.
3. Вывести результат, используя *forEach*.

```

showMales = function() { return this.gender == "m"; };
var cursor = db.unicorns.find({ $where: showMales }).sort({name: 1}).limit(2);
cursor.forEach(function(obj) {print(obj.name)})

```

```
learn> showMales = function() { return this.gender == "m"; };
[Function: showMales]
learn> var cursor = db.unicorns.find({ $where: showMales }).sort({name: 1}).limit(2);

learn> cursor.forEach(function(obj) {print(obj.name)})
Dunx
Horny
```

Практическое задание 3.2.1:

Вывести количество самок единорогов весом от полутонны до 600 кг.

```
db.unicorns.find({gender: 'f', weight: {$gte: 500, $lte: 600}}).count()
```

```
learn> db.unicorns.find({gender: 'f', weight: {$gte: 500, $lte: 600}}).count()
2
```

Практическое задание 3.2.2:

Вывести список предпочтений.

```
db.unicorns.distinct("loves")
```

```
learn> db.unicorns.distinct("loves")
[
  'apple',      'carrot',
  'chocolate', 'energon',
  'grape',      'lemon',
  'papaya',     'redbull',
  'strawberry', 'sugar',
  'watermelon'
]
```

Практическое задание 3.2.3:

Посчитать количество особей единорогов обоих полов.

```
db.unicorns.aggregate([{$group: {_id: "$gender", count: {$sum: 1}}]})
```

```
learn> db.unicorns.aggregate([{$group: {_id: "$gender", count: {$sum: 1}}]})
[ { _id: 'm', count: 7 }, { _id: 'f', count: 5 } ]
```

Практическое задание 3.3.1:

1. *Выполнить команду:*

```
> db.unicorns.save({name: 'Barney', loves: ['grape'],
weight: 340, gender: 'm'})
```

2. Проверить содержимое коллекции `unicorns`.

Функция `save` была удалена в `mongosh`, который я использовал

```
learn> db.unicorns.save({name: 'Barney', loves: ['grape'], weight: 340, gender: 'm'})
TypeError: db.unicorns.save is not a function
```

Практическое задание 3.3.2:

1. Для самки единорога `Ayna` внести изменения в БД: теперь ее вес 800, она убила 51 вампира.
2. Проверить содержимое коллекции `unicorns`.

```
db.unicorns.update({name: 'Ayna', gender: 'f'}, {$set: {weight: 800, vampires: 51}})
db.unicorns.find({name: 'Ayna'})
```

```
learn> db.unicorns.update({name: 'Ayna', gender: 'f'}, {$set: {weight: 800, vampires: 51}})
DeprecationWarning: Collection.update() is deprecated. Use updateOne, updateMany, or bulkWrite.
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn> db.unicorns.find({name: 'Ayna'})
[
  {
    _id: ObjectId('6a1756eca6e0baca51abc119'),
    name: 'Ayna',
    loves: [ 'strawberry', 'lemon' ],
    weight: 800,
    gender: 'f',
    vampires: 51
  }
]
learn> |
```

Практическое задание 3.3.3:

1. Для самца единорога `Raleigh` внести изменения в БД: теперь он любит рэдбул.
2. Проверить содержимое коллекции `unicorns`.

```
db.unicorns.update({name: 'Raleigh'}, {$set: {loves: ['redbull']}})
db.unicorns.find({name: 'Raleigh'})
```



```

learn> db.unicorns.update({name: 'Raleigh'}, {$set: {loves: ['redbull']}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn> db.unicorns.find({name: 'Raleigh'})
[
  {
    _id: ObjectId('6a1756f2a6e0baca51abc11b'),
    name: 'Raleigh',
    loves: [ 'redbull' ],
    weight: 421,
    gender: 'm',
    vampires: 2
  }
]
learn> |

```

Практическое задание 3.3.4:

1. Всем самцам единорогов увеличить количество убитых вампиров на 5.
2. Проверить содержимое коллекции `unicorns`.

```
db.unicorns.update({gender: 'm'}, {$inc: {vampires: 5}}, {multi: true})
```

```
db.unicorns.find({gender: 'm'}, {name:1, vampires:1})
```

```

learn> db.unicorns.update({gender: 'm'}, {$inc: {vampires: 5}}, {multi: true})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 7,
  modifiedCount: 7,
  upsertedCount: 0
}
learn> db.unicorns.find({gender: 'm'}, {_id:0, name:1, vampires:1})
[
  { name: 'Horny', vampires: 68 },
  { name: 'Unicrom', vampires: 187 },
  { name: 'Roooooodles', vampires: 104 },
  { name: 'Kenny', vampires: 44 },
  { name: 'Raleigh', vampires: 7 },
  { name: 'Pilot', vampires: 59 },
  { name: 'Dunx', vampires: 170 }
]

```


Практическое задание 3.3.5:

1. Изменить информацию о городе Портланд: мэр этого города теперь беспартийный.
2. Проверить содержимое коллекции `towns`.

```
db.towns.update({name: "Portland"},{$unset: {"mayor.party": 1}})
```

```
db.towns.find({name: "Portland"})
```

```
learn> db.towns.update({name: "Portland"}, {$unset: {"mayor.party": 1}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn> db.towns.find({name: "Portland"})
[
  {
    _id: ObjectId('6a175b7c2f0fef2858abc116'),
    name: 'Portland',
    population: 528000,
    last_census: ISODate('2009-07-20T00:00:00.000Z'),
    famous_for: [ 'beer', 'food' ],
    mayor: { name: 'Sam Adams' }
  }
]
```

Практическое задание 3.3.6:

1. Изменить информацию о самце единорога Pilot: теперь он любит и шоколад.
2. Проверить содержимое коллекции `unicorns`.

```
db.unicorns.update({name: 'Pilot'},{$addToSet: {loves: 'chocolate'}})
```

```
db.unicorns.find({name: 'Pilot'})
```

```
learn> db.unicorns.update({name: 'Pilot'}, {$push: {loves: 'chocolate'}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn> db.unicorns.find({name: 'Pilot'})
[
  {
    _id: ObjectId('6a1756f8a6e0baca51abc11d'),
    name: 'Pilot',
    loves: [ 'apple', 'watermelon', 'chocolate' ],
    weight: 650,
    gender: 'm',
    vampires: 59
  }
]
```

Практическое задание 3.3.7:

1. Изменить информацию о самке единорога Aurora: теперь она любит еще и сахар, и лимоны.
2. Проверить содержимое коллекции unicorns.

```
db.unicorns.update({name: 'Aurora'}, {$addToSet: {loves: {$each: ['sugar', 'lemon']}}})
db.unicorns.find({name: 'Aurora'})
```

```
learn> db.unicorns.update({name: 'Aurora'}, {$addToSet: {loves: {$each: ['sugar', 'lemon']}}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn> db.unicorns.find({name: 'Aurora'})
[
  {
    _id: ObjectId('6a1756c2a6e0baca51abc115'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape', 'sugar', 'lemon' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  }
]
```

Практическое задание 3.4.1:

1. Создайте коллекцию towns, включающую следующие документы:

```
{name: "Punxsutawney ",
popujatiuon: 6200,
last_sensus: ISODate("2008-01-31"),
famous_for: ["phil the groundhog"],
mayor: {
  name: "Jim Wehrle"
}}

{name: "New York",
popujatiuon: 22200000,
last_sensus: ISODate("2009-07-31"),
famous_for: ["status of liberty", "food"],
mayor: {
  name: "Michael Bloomberg",
  party: "I"}}

{name: "Portland",
popujatiuon: 528000,
last_sensus: ISODate("2009-07-20"),
famous_for: ["beer", "food"],
mayor: {
  name: "Sam Adams",
  party: "D"}}
```

2. Удалите документы с беспартийными мэрами.
3. Проверьте содержание коллекции.

4. *Очистите коллекцию.*
5. *Просмотрите список доступных коллекций.*

```
db.towns.remove({"mayor.party": {$exists: false}})
db.towns.find()
db.towns.remove({})
show collections
```

```
learn> db.towns.drop()
true
learn> db.towns.insert({
|   name: "Punxsutawney",
|   population: 6200,
|   last_census: ISODate("2008-01-31"),
|   famous_for: ["phil the groundhog"],
|   mayor: { name: "Jim Wehrle" }
| })
{
  acknowledged: true,
  insertedIds: { '0': ObjectId('6a1762dc2f0fef2858abc117') }
}
learn> db.towns.insert({
|   name: "New York",
|   population: 22200000,
|   last_census: ISODate("2009-07-31"),
|   famous_for: ["status of liberty", "food"],
|   mayor: { name: "Michael Bloomberg", party: "I" }
| })
{
  acknowledged: true,
  insertedIds: { '0': ObjectId('6a1762df2f0fef2858abc118') }
}
learn> db.towns.insert({
|   name: "Portland",
|   population: 528000,
|   last_census: ISODate("2009-07-20"),
|   famous_for: ["beer", "food"],
```

```

learn> db.towns.remove({"mayor.party": {$exists: false}})
DeprecationWarning: Collection.remove() is deprecated. Use de
{ acknowledged: true, deletedCount: 1 }
learn> db.towns.find()
[
  {
    _id: ObjectId('6a1762df2f0fef2858abc118'),
    name: 'New York',
    population: 22200000,
    last_census: ISODate('2009-07-31T00:00:00.000Z'),
    famous_for: [ 'status of liberty', 'food' ],
    mayor: { name: 'Michael Bloomberg', party: 'I' }
  },
  {
    _id: ObjectId('6a1762e52f0fef2858abc119'),
    name: 'Portland',
    population: 528000,
    last_census: ISODate('2009-07-20T00:00:00.000Z'),
    famous_for: [ 'beer', 'food' ],
    mayor: { name: 'Sam Adams', party: 'D' }
  }
]
learn> db.towns.remove({})
{ acknowledged: true, deletedCount: 2 }
learn> show collections
towns
unicorns
learn> |

```

Практическое задание 4.1.1:

1. Создайте коллекцию зон обитания единорогов, указав в качестве идентификатора кратко название зоны, далее включив полное название и описание.
2. Включите для нескольких единорогов в документы ссылку на зону обитания, используя второй способ автоматического связывания.
3. Проверьте содержание коллекции единорогов.

```

db.unicorns.update({name: 'Horny'},{$set: {habitat: {$ref: "habitats", $id: "forest"}}})
db.unicorns.update({name: 'Aurora'},{$set: {habitat: {$ref: "habitats", $id: "meadow"}}})
db.unicorns.update({name: 'Unicrom'},{$set: {habitat: {$ref: "habitats", $id: "mountain"}}})
db.unicorns.find({habitat: {$exists: true}}, {name:1, habitat:1})

```

```

learn> db.habitats.insert({_id: "forest", fullName: "Big Forest", description: "A really big forest"})
{ acknowledged: true, insertedIds: { '0': 'forest' } }
learn> db.habitats.insert({_id: "mountain", fullName: "High Mountains", description: "Really high mountains"})
{ acknowledged: true, insertedIds: { '0': 'mountain' } }
learn> db.habitats.insert({_id: "meadow", fullName: "Beautiful Meadow", description: "A really beautiful meadow"})
{ acknowledged: true, insertedIds: { '0': 'meadow' } }
learn> db.unicorns.update({name: 'Horny'}, {$set: {habitat: {$ref: "habitats", $id: "forest"}}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn> db.unicorns.update({name: 'Aurora'}, {$set: {habitat: {$ref: "habitats", $id: "meadow"}}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn> db.unicorns.update({name: 'Unicrom'}, {$set: {habitat: {$ref: "habitats", $id: "mountain"}}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn> db.unicorns.find({habitat: {$exists: true}}, {_id:0, name:1, habitat:1})
[
  { name: 'Horny', habitat: DBRef('habitats', 'forest') },
  { name: 'Aurora', habitat: DBRef('habitats', 'meadow') },
  { name: 'Unicrom', habitat: DBRef('habitats', 'mountain') }
]

```

Практическое задание 4.2.1:

Проверьте, можно ли задать для коллекции `unicorns` индекс для ключа `name` с флагом `unique`.

```
db.unicorns.createIndex({name: 1}, {unique: true})
```

```

learn> db.unicorns.createIndex({name: 1}, {unique: true})
name_1

```

Как видим индекс успешно был создан

Практическое задание 4.3.1:

1. Получите информацию о всех индексах коллекции `unicorns`.
2. Удалите все индексы, кроме индекса для идентификатора.
3. Попытайтесь удалить индекс для идентификатора.

```

db.unicorns.getIndexes()
db.unicorns.dropIndex({name: 1})
db.unicorns.dropIndex({_id: 1})

```



```
learn> db.unicorns.getIndexes()
[
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { name: 1 }, name: 'name_1', unique: true }
]
learn> db.unicorns.dropIndex({name: 1})
{ nIndexesWas: 2, ok: 1 }
learn> db.unicorns.dropIndex({_id: 1})
MongoServerError[InvalidOptions]: cannot drop _id index
learn> |
```

При удалении индекса для идентификации вылезает ошибка

Практическое задание 4.4.1:

1. Создайте объемную коллекцию *numbers*, задействовав курсор:

```
for(i = 0; i < 100000; i++){db.numbers.insert({value: i})}
```
2. Выберите последних четыре документа.
3. Проанализируйте план выполнения запроса 2. Сколько потребовалось времени на выполнение запроса? (по значению параметра *executionTimeMillis*)
4. Создайте индекс для ключа *value*.
5. Получите информацию о всех индексах коллекции *numbers*.
6. Выполните запрос 2.
7. Проанализируйте план выполнения запроса с установленным индексом. Сколько потребовалось времени на выполнение запроса?
8. Сравните время выполнения запросов с индексом и без. Дайте ответ на вопрос: какой запрос более эффективен?

```
for (let i = 0; i < 100000; i++) {db.numbers.insert({value: i})}
db.numbers.find().sort({value: -1}).limit(4)
db.numbers.explain("executionStats").find().sort({value: -1}).limit(4)
db.numbers.createIndex({value: 1})
db.numbers.explain("executionStats").find().sort({value: -1}).limit(4)
```

```
learn> for (let i = 0; i < 100000; i++) {db.numbers.insert({value: i})}
{
  acknowledged: true,
  insertedIds: { '0': ObjectId('6a1766aa2f0fef2858aece59') }
}
learn> db.numbers.find().sort({value: -1}).limit(4)
[
  { _id: ObjectId('6a1766aa2f0fef2858aece59'), value: 99999 },
  { _id: ObjectId('6a1766aa2f0fef2858aece58'), value: 99998 },
  { _id: ObjectId('6a1766aa2f0fef2858aece57'), value: 99997 },
  { _id: ObjectId('6a1766aa2f0fef2858aece56'), value: 99996 }
]
```



```
learn> db.numbers.explain("executionStats").find().sort({value: -1}).limit(4)
{
  explainVersion: '1',
  queryPlanner: {
    namespace: 'learn.numbers',
    parsedQuery: {},
    indexFilterSet: false,
    queryHash: 'AE5753F2',
    planCacheShapeHash: 'AE5753F2',
    planCacheKey: 'AE69D0D4',
    optimizationTimeMillis: 0,
    maxIndexedOrSolutionsReached: false,
    maxIndexedAndSolutionsReached: false,
    maxScansToExplodeReached: false,
    prunedSimilarIndexes: false,
    winningPlan: {
      isCached: false,
      stage: 'SORT',
      sortPattern: { value: -1 },
      memLimit: 104857600,
      limitAmount: 4,
      type: 'simple',
      inputStage: { stage: 'COLLSCAN', nss: 'learn.numbers', direction: 'forward' }
    },
    rejectedPlans: []
  },
  executionStats: {
    executionSuccess: true,
    nReturned: 4,
    executionTimeMillis: 92,
    totalKeysExamined: 0,
    totalDocsExamined: 100000,
    executionStages: {
      isCached: false,
      stage: 'SORT',
      nReturned: 4,
      executionTimeMillisEstimate: 73,
      works: 100006,

```

```

learn> db.numbers.createIndex({value: 1})
value_1
learn> db.numbers.explain("executionStats").find().sort({value: -1}).limit(4)
{
  explainVersion: '1',
  queryPlanner: {
    namespace: 'learn.numbers',
    parsedQuery: {},
    indexFilterSet: false,
    queryHash: 'AE5753F2',
    planCacheShapeHash: 'AE5753F2',
    planCacheKey: 'AE69D0D4',
    optimizationTimeMillis: 0,
    maxIndexedOrSolutionsReached: false,
    maxIndexedAndSolutionsReached: false,
    maxScansToExplodeReached: false,
    prunedSimilarIndexes: false,
    winningPlan: {
      isCached: false,
      stage: 'LIMIT',
      limitAmount: 4,
      inputStage: {
        stage: 'FETCH',
        nss: 'learn.numbers',
        inputStage: {
          stage: 'IXSCAN',
          nss: 'learn.numbers',
          keyPattern: { value: 1 },
          indexName: 'value_1',
          isMultiKey: false,
          multiKeyPaths: { value: [] },
          isUnique: false,
          isSparse: false,
          isPartial: false,
          indexVersion: 2,
          direction: 'backward',
          indexBounds: { value: [ '[MaxKey, MinKey]' ] }
        }
      }
    },
    rejectedPlans: []
  },
  executionStats: {
    executionSuccess: true,
    nReturned: 4,
    executionTimeMillis: 0,
    totalKeysExamined: 4,
    totalDocsExamined: 4,
    executionStages: {
      isCached: false,
      stage: 'LIMIT',
      nReturned: 4,
      executionTimeMillisEstimate: 0,

```

Без индексов время на выполнение составило примерно 92 миллисекунды

С индексом этот же запрос занял примерно 0 миллисекунд

Выводы:

В ходе лабораторной работы освоены практические приемы работы с документной базой данных MongoDB, добавление и выборка документов, фильтрация по различным условиям, включая вложенные объекты и массивы, управление выводом и сортировка. Выполнены операции обновления отдельных полей и массивов, агрегирование данных с подсчётом и группировкой. Реализованы связи между коллекциями через ссылки. Изучено создание индексов, наглядно показано сокращение времени выполнения запроса при использовании индексации. Учтены особенности современной оболочки mongosh. Полученные навыки позволяют эффективно проектировать и реализовывать запросы в среде MongoDB.